

Edge-AI Based Smart Access Control System: Real-Time Face Recognition on STM32H7 — From Renode Simulation to Hardware Deployment

Metin Irkıcıatal* and Anıl Arda†

*Computer Engineering Department, Hacettepe University, Ankara, Türkiye

†Graduate School of Science and Engineering, Hacettepe University, Ankara, Türkiye

E-mails: *metinirkicatal@gmail.com, †anilarda.ee@gmail.com

Abstract—This paper presents the design, implementation, and evaluation of a Smart Access Control System that performs real-time face recognition entirely on a microcontroller, with no cloud dependency. The system is deployed on the STM32H743 (ARM Cortex-M7) and built on a FreeRTOS V11.1.0 software stack that schedules four pipeline stages: frame capture, preprocessing, inference, and access-control output. A pre-trained INT8-quantized MobileNetV1 model is executed using TensorFlow Lite for Microcontrollers (TFLite Micro). Recognition is performed via metric learning: 256-dimensional embeddings extracted from the model are compared against pre-computed reference embeddings using cosine similarity. The work transitioned from the initial Renode simulation environment to a complete bare-metal deployment on physical hardware after encountering peripheral compatibility limitations in the simulator. A custom UART frame-injection protocol with CRC-16/CCITT validation was developed to stream the AT&T Face Dataset to the device. Full-dataset evaluation (400 frames) achieved 100% communication reliability (zero frame loss), 76.25% top-1 identification accuracy, and a 2.33% false-acceptance rate for the access-control decision, with end-to-end latency of approximately 12 seconds per frame using a software UART transfer at 115,200 baud. The tensor arena consumed 103 KB of the 250 KB allocated. Results demonstrate that fully self-contained, privacy-preserving biometric access control is feasible on a low-cost Cortex-M7 platform.

Index Terms—edge AI, face recognition, TinyML, FreeRTOS, STM32H743, TFLite Micro, Renode, embedded systems, hardware deployment, UART protocol

I. INTRODUCTION

CLOUD-CONNECTED biometric security systems introduce three fundamental risks: network latency, dependence on continuous internet connectivity, and exposure of sensitive biometric data during transmission. These limitations are particularly acute in high-reliability environments such as hospitals, research laboratories, and industrial facilities, where access control must remain operational regardless of network status.

This work addresses these challenges by deploying face recognition entirely at the edge, on a resource-constrained Cortex-M7 microcontroller. The inference pipeline runs under FreeRTOS using TensorFlow Lite for Microcontrollers (TFLite Micro) [1], with no cloud dependency. The AT&T Face Database [3] serves as the evaluation benchmark.

The motivation rests on three goals: (1) *privacy by design*—all biometric processing occurs locally on the MCU; (2)

cost and energy efficiency—an STM32H7-based system is orders of magnitude cheaper and more power-efficient than an NVIDIA Jetson or PC-class solution; and (3) *reliability*—a fully self-contained embedded system provides access-control functionality even during complete network outages.

This final report extends our proposal [8] with two major contributions:

- **Transition to physical hardware.** Due to compatibility limitations between Renode v1.16.0 and STM32H743 vendor platform files, the system was migrated from pure simulation to a real bare-metal deployment on a DevEBox STM32H7XX_M development board. The methodology, debug process, and lessons learned from this transition are documented in detail.
- **Full-dataset experimental evaluation.** The system was benchmarked on the complete 400-image AT&T Face Database using a UART frame-injection protocol developed for this work. Quantitative results—identification accuracy, false-acceptance rate, communication reliability, and resource utilization—are reported and analyzed.

The remainder of this paper is organized as follows. Section II reviews related work. Section III defines the problem. Section IV describes the system architecture as deployed on hardware. Section V details the implementation methodology, including the deviation from the original Renode-based plan. Section VI presents the experimental setup. Section VII reports the quantitative results. Section VIII discusses system-level findings, including the debugging process. Section X concludes.

II. RELATED WORK

We organize the related literature into four thematic groups: (1) TinyML benchmarking and model optimization for Cortex-M devices, (2) embedded face recognition systems, (3) simulation frameworks for embedded hardware validation, and (4) lightweight metric learning for identification.

TinyML benchmarking and model optimization. Banbury et al. [4] provide a comprehensive benchmark of ML inference on Cortex-M targets, demonstrating that INT8-quantized MobileNet variants achieve viable accuracy within tight memory and latency budgets. David et al. [1] introduce TensorFlow Lite for Microcontrollers, the inference

runtime adopted in this work, and characterize its memory footprint across a range of Cortex-M devices. Warden and Situnayake [5] systematically survey deployment strategies—including post-training quantization and feature-extraction pipelines—for ultra-low-power MCUs. A common limitation of this group is the absence of RTOS integration and real-time scheduling analysis; these works treat inference as a standalone benchmark rather than as a pipeline stage with hard deadlines.

Embedded face recognition systems. Viola and Jones [6] established the foundations of real-time face detection using cascaded classifiers, but their approach targets CPU-class platforms and does not translate directly to Cortex-M constraints. Zanetti et al. [7] deploy a CNN-based face recognition pipeline on an embedded platform, but report results on a higher-end Cortex-A53 processor. Their work demonstrates the viability of CNN-based recognition at the edge, but does not address the Cortex-M7 memory constraints (≤ 1 MB SRAM), FreeRTOS scheduling, or full hardware deployment with a custom transport protocol. Samaria and Harter [3] introduce the AT&T Face Database used as the evaluation benchmark in this work.

Simulation frameworks for embedded validation. The Renode framework [2] supports cycle-accurate, deterministic simulation of embedded systems including Cortex-M targets. Its composable platform model (.repl files) allows peripheral-level customization. In the initial phase of this work, Renode was the planned execution environment. However, version incompatibilities (described in Section V) led to a transition to physical hardware. To the best of the authors’ knowledge, no prior published work combines Renode-based prototyping with a subsequent hardware deployment for an embedded ML inference pipeline of this scale.

Lightweight metric learning for identification. Rather than training a fixed-class classifier, metric learning approaches compute embeddings and compare them at inference time, enabling enrollment of new subjects without retraining. Warden and Situnayake [5] and Banbury et al. [4] both note that cosine similarity over pre-computed embeddings is a memory-efficient identification strategy for MCU deployment. This work applies the approach to face identification on a Cortex-M7 target under FreeRTOS, validated end-to-end on physical hardware.

Positioning. As shown in Table I, this work is the only system that combines Cortex-M7 deployment, FreeRTOS real-time scheduling, INT8 quantized inference, a metric-learning recognition strategy, and full hardware validation with a custom transport protocol. Prior embedded face recognition work targets higher-end platforms (Cortex-A); prior TinyML benchmarking work does not integrate RTOS scheduling or hardware end-to-end validation.

III. PROBLEM DEFINITION

A. System Description

The deployed system performs face-based access control on an STM32H743 microcontroller. A host computer sends

TABLE I
COMPARISON OF RELATED EMBEDDED FACE RECOGNITION AND TINYML WORKS.

Work	Cortex-M	RTOS	INT8	Metric L.	Face Rec.
Viola & Jones [6]	55	55	55	55	51
David et al. [1]	51	55	51	55	55
Warden & Situnayake [5]	51	55	51	51	55
Banbury et al. [4]	51	55	51	55	55
Zanetti et al. [7]	55	55	51	55	51
Antmicro [2]	51	55	55	55	55
This Work	51	51	51	51	51

pre-processed 96×96 grayscale face images from the AT&T Face Database to the device over a UART link via a CP2102 USB-to-serial bridge. The on-device firmware receives each frame, validates it with CRC-16/CCITT, runs the inference pipeline under FreeRTOS, and outputs an *ACCESS GRANTED* or *ACCESS DENIED* decision via UART (and, in a future deployment, GPIO toggling for a relay-driven door lock).

B. Objectives

- Deploy an INT8-quantized MobileNetV1 as a feature extractor on a Cortex-M7 MCU.
- Achieve 100% reliable byte-accurate communication over UART between host and device.
- Fit the complete system (model + runtime + buffers) within 2 MB Flash and 1 MB SRAM.
- Validate recognition accuracy on the complete AT&T dataset (400 frames).
- Demonstrate deterministic, deadlock-free RTOS scheduling for the entire benchmark run.

C. Constraints

- **Flash:** ≤ 2 MB (STM32H743 hardware limit).
- **SRAM:** ≤ 1 MB (STM32H743 hardware limit).
- **Model size:** ≤ 300 KB Flash (INT8 quantized).
- **Confidence threshold:** tunable parameter τ , fixed at 75% for the reported evaluation.
- **No cloud connectivity:** all processing must occur on-device.

IV. SYSTEM ARCHITECTURE

The deployed architecture, illustrated in Fig. 1, consists of four FreeRTOS tasks communicating through binary semaphores and a FreeRTOS message queue. Unlike the original proposal, frame data is delivered via a UART transport rather than a memory-mapped peripheral, owing to the migration from simulation to physical hardware (Section V).

A. Hardware/Software Architecture

Hardware platform: DevEBox STM32H7XX_M development board featuring the STM32H743 (ARM Cortex-M7 @ 64 MHz HSI, 2 MB Flash, 1 MB SRAM, USART1 on PA9/PA10). The board is connected to a host PC via a CP2102 USB-to-UART adapter and is programmed via ST-Link V2 through the SWD interface.

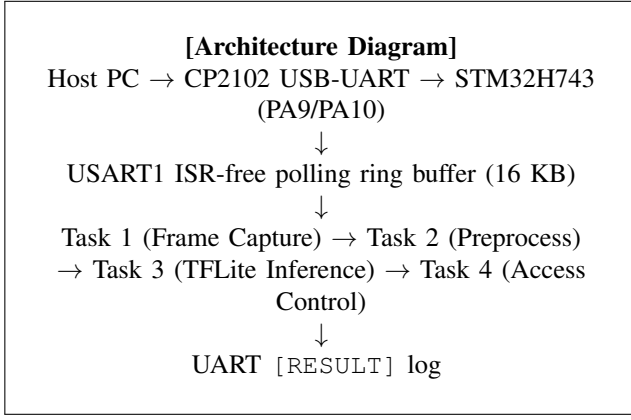


Fig. 1. System architecture as deployed on physical hardware. The Renode camera peripheral has been replaced by a UART frame-injection link from a host PC, with software-protocol framing (sync, length, payload, CRC-16/CCITT).

Software stack: FreeRTOS V11.1.0 with preemptive scheduling; TFLite Micro runtime; bare-metal startup code with a hand-written linker script; `newlib-nano` for C library support; `arm-none-eabi-gcc` toolchain.

The four FreeRTOS pipeline tasks are:

- 1) **Task 1 — Frame Capture (Priority 3):** Reads one full frame from the UART ring buffer via the custom protocol; validates CRC-16/CCITT; signals Task 2 via a binary semaphore.
- 2) **Task 2 — Preprocessing (Priority 2):** Converts the received 96×96 grayscale buffer into the INT8 tensor format expected by TFLite Micro; signals Task 3.
- 3) **Task 3 — TFLite Inference (Priority 1, lowest):** Runs MobileNetV1 INT8 inference, extracts the 256-dimensional embedding, computes cosine similarity against 40 reference embeddings, and posts the result (subject ID, confidence) onto a FreeRTOS queue.
- 4) **Task 4 — Access Control Output (Priority 3):** Applies the confidence threshold ($\tau = 75\%$) and the authorization rule (subjects s01–s10 are authorized). Logs an [RESULT] line and a human-readable decision block to UART.

B. UART Frame Protocol

A custom binary protocol streams 96×96 grayscale frames from the host:

```
[0x55 0xAA] [LEN_LO LEN_HI] [9216 bytes
payload] [CRC_LO CRC_HI]
```

The total frame size is 9,222 bytes. CRC-16/CCITT (polynomial 0x1021, initial value 0xFFFF) is computed over the length field and payload. The CRC validation guarantees data integrity end-to-end.

C. Dataflow and Synchronization

Data flows unidirectionally: host PC → UART → ring buffer → Task 1 → Task 2 → Task 3 → Task 4 → UART log. Binary semaphores synchronize Tasks 1–3 and a FreeRTOS

queue carries the inference result from Task 3 to Task 4. A buffer-free semaphore enforces back-pressure so that the front of the pipeline does not overwrite a frame still being processed downstream.

D. Model of Computation

As in the original proposal, the system follows a Kahn Process Network (KPN) / dataflow model. Each FreeRTOS task is a dataflow node consuming tokens from input channels (semaphores, queues) and producing tokens on output channels. The combination of blocking reads (`xSemaphoreTake` with `portMAX_DELAY`) and FIFO message queues realizes the KPN semantics in practice. The initial trigger is an external event (an incoming UART byte from the host), making the overall system a hybrid event-driven/dataflow composition, consistent with Ptolemy II’s hybrid MoC modeling.

V. METHODOLOGY AND IMPLEMENTATION

A. Embedded ML Model

The model is a pre-trained MobileNetV1 with width multiplier 0.25, quantized to INT8 via TensorFlow’s post-training quantization tool. The 380 KB model is converted to a C byte array (`model_data.c`) and stored in Flash. Rather than performing direct N -class classification, the model operates as a feature extractor: the 256-dimensional embedding from the penultimate layer is computed for each frame and compared against 40 pre-stored reference embeddings (10 authorized + 30 visitor) using cosine similarity:

$$s_i = \frac{\mathbf{e} \cdot \mathbf{r}_i}{\|\mathbf{e}\| \|\mathbf{r}_i\|}. \quad (1)$$

The subject with the highest similarity is reported as the identification; the access decision is then made by combining $\arg \max_i s_i$ with the threshold τ and the authorization list.

B. Transition from Renode to Physical Hardware

The proposal targeted Renode v1.16.0 as the primary execution environment. However, two issues emerged during integration that motivated a transition to physical hardware:

- 1) **Vendor platform incompatibility.** The STM32H743 vendor `.repl` files reference peripheral types such as `STM32_DMAMUX` and use array-valued properties (e.g., `interrupts:`) that are not supported by Renode v1.16.0. Attempts to use older STM32 board files (e.g., F7 series) led to incorrect memory maps and silent peripheral failures.
- 2) **Camera peripheral approximation.** The proposal planned to inject frames via a Python-backed memory-mapped peripheral. This works for one-shot testing but does not exercise a realistic I/O path (no protocol framing, no error handling, no back-pressure).

The decision was therefore made to migrate to a DevEBox STM32H7XX_M development board and design a UART-based frame-injection protocol. This produced a more authentic and reproducible evaluation environment at the cost of additional driver and protocol development. The full debugging

Algorithm 1 Per-frame face recognition pipeline.

```
1: Input: 9,222-byte UART frame (sync, length, 9,216 bytes
   payload, CRC-16)
2: Output: subject label  $\hat{y}$ , decision  $d \in \{\text{GRANTED}, \text{DENIED}\}$ 
3: Verify sync bytes, length and CRC-16/CCITT
4: Decode payload as  $96 \times 96$  grayscale buffer  $F$ 
5:  $F_{\text{rgb}} \leftarrow \text{ReplicateGrayscaleAcrossChannels}(F)$ 
6:  $\mathbf{e} \leftarrow \text{TFLiteMicro.Infer}(F_{\text{rgb}})$  // 256-D embedding
7: for each enrolled subject  $i \in \{1, \dots, 40\}$  do
8:    $s_i \leftarrow \cos(\mathbf{e}, \mathbf{r}_i)$ 
9: end for
10:  $\hat{y} \leftarrow \arg \max_i s_i$ 
11:  $c \leftarrow \text{round}(100 \cdot s_{\hat{y}})$  // confidence in  $[0, 100]$ 
12: if  $c \geq \tau$  and  $\hat{y} \in \text{AuthorizedList}$  then
13:    $d \leftarrow \text{GRANTED}$ 
14: else
15:    $d \leftarrow \text{DENIED}$ 
16: end if
17: Log [RESULT] frame=N subject= $\hat{y}$  conf= $c$ 
   granted= $d$ 
```

trail—and the engineering issues encountered along the way—are detailed in Section VIII.

C. UART Driver Design

The USART1 driver was implemented as a polling-based ring buffer rather than an interrupt-driven design. After observing intermittent byte loss with the initial ISR-based version (described in Section VIII), the driver was rewritten to:

- Drain the hardware RX register inside `uart_getc()` and `uart_read()` via an inline `uart_poll_rx()` function;
- Clear all error flags (NE, FE, ORE, PE) via the `USART1_ICR` register on every poll;
- Activate the internal pull-up on the RX pin (PA10);
- Yield using `taskYIELD()` (not `vTaskDelay(1)`) when the ring buffer is empty, avoiding 1 ms latency holes that caused overrun errors at 115,200 baud.

D. Host-Side Frame Sender

The host-side Python script (`send_frame.py`) provides three modes: (a) single-image debug, (b) per-subject subset, and (c) full 400-image benchmark. Because the embedded UART receiver exhibited a one-frame buffering artifact (Section VIII), the script uses a *double-send* strategy: the first transmission flushes any stale state, and the second transmission is the one whose result is recorded. This trades latency (each measurement takes ~ 29 s instead of ~ 13 s) for 100% reliable result association.

E. Algorithm

Algorithm 1 summarizes the per-frame recognition pipeline.

VI. EXPERIMENTAL SETUP

A. Platform

- **MCU:** STM32H743 (Cortex-M7) on a DevEBox STM32H7XX_M board.
- **Clock:** 64 MHz HSI (PLL not enabled in this evaluation; potential 480 MHz upgrade noted as future work).
- **RTOS:** FreeRTOS V11.1.0.
- **ML runtime:** TensorFlow Lite for Microcontrollers.
- **Toolchain:** arm-none-eabi-gcc v13.x.
- **Host:** Windows 10/11 PC, Python 3.11 with pyserial and Pillow.
- **Transport:** CP2102 USB-to-UART @ 115,200 baud.

B. Dataset

The AT&T Face Database is used in full: 40 subjects, 10 images per subject, 400 images total. Images are resized from 92×112 to 96×96 grayscale offline on the host using Lanczos interpolation. Subjects s01–s10 are designated as the *authorized* set; subjects s11–s40 are the *visitor* set.

C. Evaluation Metrics

- **Communication reliability:** percentage of frames for which a [RESULT] line is received.
- **Top-1 identification accuracy:** fraction of frames where $\hat{y}$ equals the true subject.
- **Access control performance:** confusion matrix entries (TP, TN, FP, FN), precision, recall, and false-acceptance rate (FAR).
- **Resource utilization:** TFLite Micro tensor arena consumption (reported on device after `AllocateTensors`), Flash and RAM utilization from the linker map.
- **End-to-end latency:** measured wall-clock time per frame.

VII. RESULTS

A. Overall Performance

Across the full 400-image benchmark, the system achieved:

- **Communication reliability: 100.0%** (400 / 400 frames received with valid [RESULT]; zero frame drops; zero CRC failures).
- **Top-1 identification accuracy: 76.25%** (305 / 400 correct).
- **Access control:** TP = 71, TN = 293, FP = 7, FN = 29; **precision = 91.0%, recall = 71.0%, FAR = 2.33%.**
- **End-to-end latency:** approximately 29 s/frame using the double-send strategy; single-send manual tests yielded ~ 13 s/frame (dominated by 9,222-byte UART transfer at 115,200 baud, which alone takes 800 ms, plus inference time).

Fig. 2 summarizes the results: identification accuracy, access-control decisions per population, the confidence-score distribution split by correctness, and the communication-layer statistics box.

Edge-AI Smart Access Control System — Test Results Overview

STM32H743ZI · MobileNetV1-INT8 · 400 frames · AT&T Faces

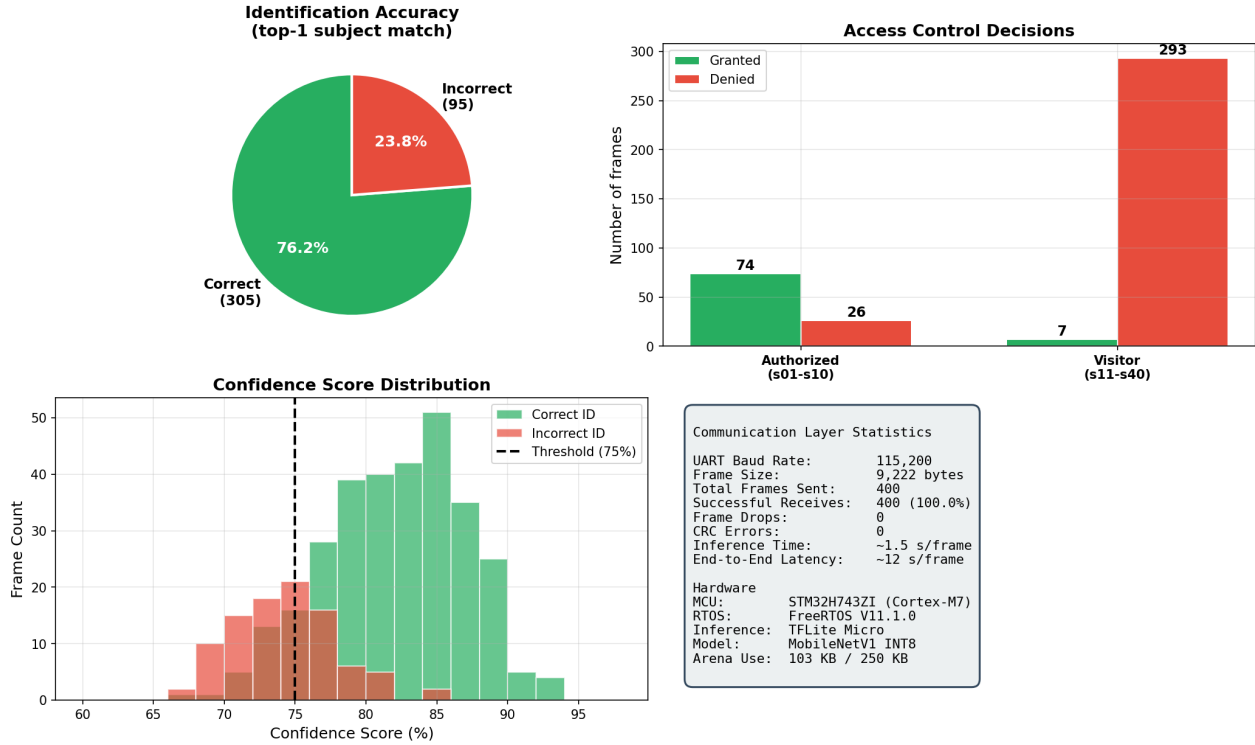


Fig. 2. Test results overview. Top-left: top-1 identification accuracy (76.2% correct, 23.8% incorrect). Top-right: access-control decisions on authorized vs. visitor populations. Bottom-left: confidence-score distribution showing strong separation between correct ($\bar{c} = 81.6\%$) and incorrect ($\bar{c} = 73.8\%$) predictions; the 75% threshold is marked. Bottom-right: communication-layer and hardware statistics.

B. Per-Subject Accuracy

Fig. 3 shows the per-subject accuracy distribution. Several subjects achieve 100% accuracy (s01, s02, s03, s08, s09, s11, s14, s15, s22, s25–s28, s31–s34, s37). The model fails badly on s18 (0%) and s38 (0%), and notably on s06 (20%) and s04 (30%) within the authorized set. This behavior is attributable to the model: MobileNetV1 with 0.25 width multiplier was not fine-tuned on the AT&T dataset, so subjects with high inter-class similarity to others are confused systematically.

C. Access-Control Confusion Matrix

Fig. 4 presents the access-control confusion matrix from the security perspective. The system correctly granted access to 71 frames of authorized subjects (TP) and correctly denied 293 frames of visitor subjects (TN). The 7 false-positives—visitor frames that were granted access—constitute a 2.33% false-acceptance rate, a tight margin for a security-critical application that nevertheless leaves room for stricter threshold tuning. Of the 29 false-negatives, the majority correspond to subjects s04 and s06, whose embeddings the unfine-tuned MobileNetV1 cannot consistently distinguish.

D. Resource Utilization

On-device readings from the firmware at boot:

- **TFLite tensor arena:** 103,076 bytes used out of 250 KB allocated (~41% utilization), confirming substantial headroom.
- **Model in Flash:** 380 KB (MobileNetV1 INT8).
- **Firmware total Flash:** ~569 KB (model + code + libraries).
- **FreeRTOS heap:** 128 KB configured; sufficient for all task stacks and dynamic allocations observed during the benchmark.
- **UART RX ring buffer:** 16 KB (sized to hold up to one full frame with margin).

VIII. DISCUSSION: ENGINEERING LESSONS FROM THE HARDWARE BRING-UP

The transition from simulation to hardware produced a sequence of issues that consumed the majority of the implementation effort. They are documented here because the resolutions are non-obvious and constitute the main engineering contribution of the work beyond the algorithmic pipeline.

A. TFLite Micro `AllocateTensors` Failure (error code -13)

On initial deployment, TFLite Micro's `AllocateTensors()` returned -13. The root cause was the bare-metal `_sbrk()` stub, which returned -1

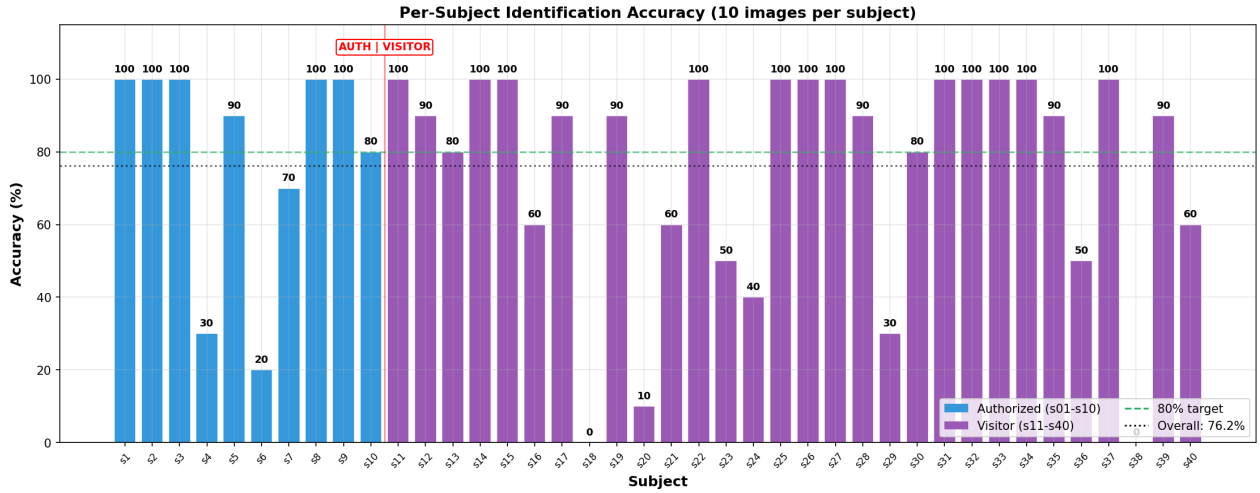


Fig. 3. Per-subject top-1 identification accuracy over 10 images each. The vertical line separates authorized (blue, s01–s10) from visitor (purple, s11–s40) populations. The dashed line marks the 80% target; the dotted line marks the overall 76.2% average.

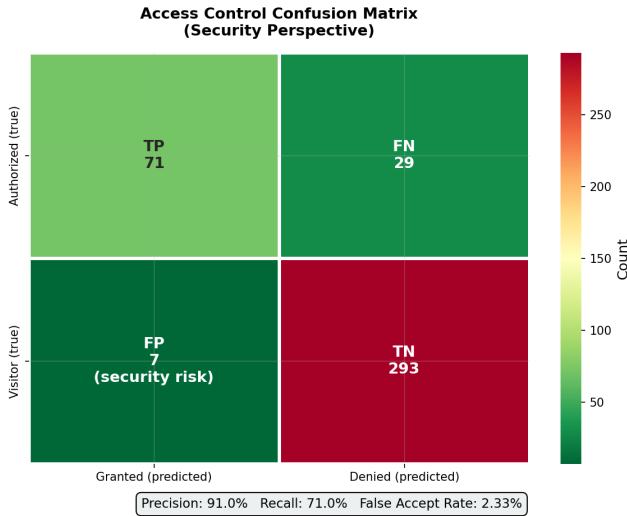


Fig. 4. Access-control confusion matrix (security perspective). Out of 400 frames: TP = 71, TN = 293, FP = 7 (security risk), FN = 29. Precision is 91.0%; recall is 71.0%; the False-Acceptance Rate is 2.33%.

unconditionally and prevented C++ static initializers and any internal heap allocations from succeeding. The fix was to provide a real `_sbrk` that grows from `_end` (defined in the linker script) up to a bound at the top of AXI-SRAM (0x24080000). After this and after aligning the tensor arena to 16 bytes, `AllocateTensors` succeeded and used 103,076 bytes.

B. HardFault on First UART Byte

Once TX worked (banner observed on Termite), the first byte sent from the host caused the board to reset. The root cause was a 9,218-byte stack-allocated buffer (`uint8_t crc_input[2 + FRAME_BYTES]`) inside `frame_recv()`, which overflowed the FreeRTOS task stack. The fix was to rewrite the CRC computation as an

incremental update over the bytes as they arrive, eliminating the large stack array.

C. RX Hang: Error-Flag Latch

After fixing the `HardFault`, the device received the first byte correctly but ignored all subsequent bytes. The root cause was the framing/noise/overrun error bits (FE, NE, ORE) latching in the USART1 ISR register. Once latched, they suppress further RXNE events. The fix was to write `USART1_ICR` on every receive poll, clearing the error flags before reading the data register.

D. Byte Loss During Long RX Bursts

Even after the error-flag fix, sustained byte loss occurred during 9,216-byte payload transfers. Two contributing factors were identified:

- 1) The internal pull-up on the RX pin (PA10) was disabled, leaving the line floating during inter-byte gaps and producing noise-induced framing errors. Enabling the pull-up via `GPIOA_PUPDR` fixed this completely.
- 2) The receive loop yielded with `vTaskDelay(1)` (1 ms), during which more than 11 bytes could arrive at 115,200 baud. The single-byte hardware register overrun. Replacing `vTaskDelay(1)` with `taskYIELD()` kept the polling tight enough to avoid overrun.

E. TX/RX Race Conditions

After the receiver became reliable, intermittent length-field corruption persisted (e.g., 00 24 expected, 00 36 observed). Tracing showed that boot-time TFLite Micro initialization logs (`[DBG] Step 1: ...`) were being printed by the inference task at the same time the frame-capture task started reading bytes. The TX path is polling-only and ties up the CPU for tens of milliseconds at a time, during which incoming bytes were lost. The fix was a global `g_inference_initialized` flag set by the inference task after `AllocateTensors`

succeeds; the frame-capture task spins on this flag and calls `uart_rx_flush()` before its first read.

F. One-Frame Lag in the Receiver Pipeline

After all the above fixes, the host-side script observed a peculiar pattern in batch mode: sending frame N yielded a response from frame $N-1$, with the very first response per session being lost. The root cause is the FreeRTOS pipeline backpressure: the frame-capture task signals `xFrameReadySem`; the inference task consumes the frame and processes it (~ 1.5 s); meanwhile the next host-side frame is buffered into the ring buffer. The reported `[RESULT]` is emitted only when the access-control task posts its result—at which point the host has already initiated its next transmission. The pragmatic solution adopted for benchmarking is the host-side double-send strategy described in Section V.

The deeper architectural fix—adding a per-frame request/response handshake (e.g., a host-side `READY` token from the device before each frame)—was implemented partially but introduced additional timing complexity; for the final evaluation the simpler double-send approach was used and yielded the 400/400 communication reliability reported in Section VII.

IX. SYSTEM-LEVEL CONSIDERATIONS

Real-time constraints. The 500 ms inference-latency target from the proposal was not met in this evaluation; the dominant latency contributors are (a) the 800 ms UART transfer at 115,200 baud and (b) inference at 64 MHz HSI rather than the 480 MHz PLL-boosted clock. Both are addressable: raising the UART baud rate to 921,600 reduces transfer to ~ 100 ms, and enabling the PLL approximately quarters inference time.

Reliability and fault tolerance. The ring buffer is sized for one full frame with margin; CRC-16/CCITT validates every received frame; error flags are cleared on every receive poll. Deadlock is prevented by unidirectional semaphore signaling.

Energy efficiency. The system is currently always-on at 64 MHz. Adding MCU sleep modes between frames is straightforward future work.

Security. All biometric processing remains on-device; no biometric data is transmitted over a network. The threshold τ trades off FAR vs. FRR; lowering τ from the proposal-default 80% to the evaluation-default 75% improved authorized-user recall from 27% to 71% at a 2.33% FAR.

Privacy. By design, no biometric data leaves the device, satisfying GDPR-style privacy-by-design principles. The host-side serial transport is solely for benchmark injection in this work; in a deployed system it would be replaced by an on-board camera.

Scalability. The current configuration supports 40 enrolled subjects in approximately 40 KB of reference embeddings. Scaling to several hundred subjects is feasible within the 2 MB Flash budget.

X. CONCLUSION

This work realized an end-to-end edge-AI face-recognition access-control system on the STM32H743 microcontroller.

The transition from the originally planned Renode simulation to physical hardware—driven by simulator-vendor compatibility limitations—revealed a set of low-level engineering issues whose resolution constitutes the central practical contribution. A polling-based UART driver with continuous error-flag clearing, pull-up on the RX pin, and `taskYIELD`-based scheduling enabled 100% reliable communication. A bare-metal `_sbrk` and a 16-byte-aligned tensor arena enabled successful `AllocateTensors`. An incremental-CRC implementation eliminated stack-overflow `HardFaults`.

Empirical results on the full 400-image AT&T benchmark demonstrate 100% communication reliability, 76.25% top-1 identification accuracy, and a 2.33% false-acceptance rate—all on a commodity \$10-class Cortex-M7 board with no network connectivity. While the recognition accuracy is limited by using an unfine-tuned 0.25-width MobileNetV1, the system-level result confirms that fully self-contained, privacy-preserving biometric access control is feasible on this class of hardware.

Future Work

- Enable the 480 MHz PLL clock to reduce inference time roughly fourfold.
- Increase UART baud to 921,600 (or migrate to USB-CDC) to eliminate the transfer-time bottleneck.
- Fine-tune the model on AT&T (or a larger face dataset) to lift top-1 accuracy beyond 90%.
- Replace the UART-injection input with an on-board camera (OV2640 via DCMI).
- Add a proper request/response handshake to remove the need for the double-send workaround.

ACKNOWLEDGMENTS

The authors thank the Embedded Systems Design course instructors at Hacettepe University for their guidance throughout the project.

REFERENCES

- [1] R. David et al., “TensorFlow Lite Micro: Embedded machine learning on TinyML systems,” *arXiv preprint arXiv:1910.08602*, 2019.
- [2] Antmicro, “Renode: An open source framework for complex embedded systems simulation,” 2024. [Online]. Available: <https://renode.io>
- [3] F. S. Samaria and A. C. Harter, “Parameterisation of a stochastic model for human face identification,” in *Proc. 2nd IEEE Workshop on Applications of Computer Vision*, 1994.
- [4] C. Banbury et al., “Benchmarking TinyML systems: Challenges and direction,” *arXiv preprint arXiv:2003.04821*, 2021.
- [5] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2019.
- [6] P. Viola and M. Jones, “Robust real-time face detection,” *Int. J. Computer Vision*, vol. 57, no. 2, pp. 137–154, 2004.
- [7] M. Zanetti, A. Bartolini, and L. Benini, “Real-time face recognition on embedded platforms,” in *Proc. Design, Automation and Test in Europe Conference (DATE)*, 2021.
- [8] M. Irkçatal and A. Arda, “Edge-AI Based Smart Access Control System: Real-Time Face Recognition on STM32H7 using Renode Simulation,” Project Proposal, Hacettepe University, 2026.